

正则表达式

主要内容

- 正则表达式
- 模式对象和相应方法
- 子模式扩展(可选的内容)
- [Regular-Expressions.info - Regex Tutorial, Examples and Reference - Regexp Patterns](#)
- [regex101: build, test, and debug regex](#)
- [Debuggex: Online visual regex tester. JavaScript, Python, and PCRE.](#)

正则表达式(Regular Expression)

- 字符串已经有：
 - `in` 判断子字符串是否出现: '213-867-5309' in 'My phone number : 213-867-5309.'
 - `find(sub [,index,[stop]])` 和 `index(sub....)` 判断子字符串sub出现的位置
 - `split(sep, maxsplit)`: 分割字符串
 - `replace(old, new [,count])` 用new替代其中的子串old
 - `maketrans(x, y, z)` `translate()` 以单字符为单位进行替代或删除
- 如果想要知道字符串里面是否包括合法的电话号码, 以及这些号码是什么?
 - 电话号码满足一定的**模式**, str只支持**固定的字符串**的查找
 - 引入正则表达式, 描述模式(pattern)
 - 可以判断某个**模式**是否在字符串中出现 `matchobj = re.search(pattern)`
 - 可以知道**模式在字符串出现的位置以及匹配的内容** `matchobj.span()`
`matchobj.group()`
 - 可以根据模式**分割字符串**, 可以根据模式**替换字符串** `re.split(...)` `re.sub(...)`

正则表达式

- 正则表达式：由普通字符和具有特殊含义的元字符(meta-character)组成的模式(pattern)
 - 最简单的正则表达式仅仅包含普通的字符，比如 'for'
- re模块(import re)提供了使用正则表达式进行查找、替代和分割的方法
 - re.findall(pattern, string, flags=0)
 - 在字符串string中搜索与模式pattern匹配的内容，接下来继续搜索下一个匹配...，找到的匹配组成一个**列表**返回
 - flags给出了搜索时正则表达式匹配的选项，比如是否大小写无关(re.I或re.IGNORECASE)等

```
>>> import re
>>> text='The quick brown fox jumped for food. The lazy black dog jumped for FOOD.'
>>> re.findall('for', text)
['for', 'for']
>>> re.findall('food', text)
['food']
>>> re.findall('food', text, re.I) # 匹配时大小写无关
['food', 'FOOD']
```

正则表达式

`regex = re.compile(pattern, flags=0)`

- 将字符串`pattern`中的模式转换（编译）成内部的表示形式，返回正则表达式(或模式)对象(类型为`re.Pattern`)
- `flags`可选，给出了使用该模式对象进行匹配时所采用的选项
- 模式对象支持`findall`等方法：`regex.findall(string[, pos[, endpos]])` 在`string`指定位置查找`regex.pattern` (`regex.flags`给出了匹配的选项)

`re.findall(pattern, string, flags) ⇔ re.compile(pattern, flags).findall(string)`

```
>>> import re
>>> text='The quick brown fox jumped for food. The lazy black dog jumped for FOOD.'
>>> print(f'{type(regex)}, {regex.pattern = }')
<class 're.Pattern'>, regex.pattern = 'food'
>>> regex = re.compile('food')
>>> regex.findall(text)
['food']
```

re模块的方法

```
import re
text = 'The quick brown fox jumped for food. The lazy black dog jumped for FOOD.'
results = re.findall('food', text, re.I) # 匹配时大小写无关

regex = re.compile('food', re.I)
results = regex.findall(text)
```

到底采用哪种方法呢？

- 如果某个pattern只需要匹配一次，两种方法都可以
- 如果某个pattern要匹配多次，建议首先re.compile得到一个模式对象regex，然后每次匹配使用regex.findall(text)
 - 节省多次函数调用（不需要每次调用compile...）

re.findall(.)

findall: pattern + text

re.compile(.).findall(.)

findall: pattern + text

findall: pattern + text

← findall: pattern + text →

匹配选项

- Pattern和string都是str对象, 选项缺省为re.U
- Pattern和string都是bytes对象, 选项缺省为re.L

`re.findall(pattern, string, flags=0)` `re.compile(pattern, flags=0)`

- flags: 匹配选项。传递的值为0时表示采用缺省的匹配选项 (re.U)
- 可以采用短格式(如re.I), 也可采用长格式(如re.IGNORECASE)
- 多个选项可以通过运算符 | (按位或) 组合, 比如 re.I | re.U

```
>>> import re
>>> regexp = re.compile('food')
>>> regexp
re.compile(r'food', re.UNICODE)
>>> regexp = re.compile('food', re.I)
>>> regexp
re.compile(r'food', re.IGNORECASE|re.UNICODE)
>>> regexp = re.compile('food', re.I | re.MULTILINE)
>>> regexp
re.compile(r'food', re.IGNORECASE|re.MULTILINE|re.UNICODE)
```

匹配选项(flags)	说明
re.I re.IGNORECASE	忽略大小写
re.L re.LOCALE	\w \W \b \B \d \D \s \S与本地字符集有关, re模块用于bytes对象时使用
re.U re.UNICODE	\w \W \b \B \d \D \s \S的定义基于Unicode, pattern为str时缺省选项
re.A re.ASCII	\w \W \b \B \d \D \s \S的定义基于ASCII
re.S re.DOTALL	元字符.(dot) 也匹配换行符
re.X re.VERBOSE	忽略模式中的空格 (这样可以写成多行, 也可添加空格), 并可以使用#注释, 提高可读性
re.M re.MULTILINE	多行匹配模式, ^ \$可以匹配每行的开始和结尾

字符类	[set of characters]	匹配中括号里面枚举出的多个字符中的任一字符。第一个字符为^表示否定，-在中间出现表示范围，\表示转义。其他元字符都失去其含义，表示字符本身	[abc] [[abc] [-abc^\n\]]
	-	[]内部使用，描述指定范围的字符。如果出现在最初或最后，匹配连字符本身	[0-9a-fA-F] [a-] [-ab]
	^set of characters	否定枚举，匹配不在后面描述的字符集中的字符。^出现在后面时匹配^本身	[^xyz] [^a-zA-Z]
	\d \D \s \S \w \W	预定义字符类，匹配数字类,非数字类,空格类,非空格类,单词类,非单词类字符	[\d\s] \shelllo\s
	.(dot)	匹配除换行符以外的任意单个字符。如果re.S，也匹配换行符	^\s*#.*\$ 匹配注释行
重复	X{n,m}	表示前面的字符或子模式X重复的次数(n-m次)，其中n或m可以省略	\d{1,3} \d{,3} (fo){1,}
	X{n}	表示前面的字符或子模式X重复正好n次	\d{3}
	X+	表示前面的字符或子模式X重复1次以上，相当于X{1,}	\d+
	X*	表示前面的字符或子模式X重复0次以上，相当于X{0,}	\d*
	X?	表示前面的字符或子模式X重复0次或1次，相当于X{0,1}	\d?
边界	^ \A	匹配字符串的开始位置，多行模式下还匹配每行的行首。 \A表示仅匹配开始位置	^The
	\$ \Z	匹配字符串的结束位置或字符串最后一行的行尾（即最后位置为换行符时其之前的位置）。多行模式下还匹配每行的行尾。 \Z表示仅匹配结束位置	food\$
	\b \B	匹配单词的开始或者结束位置， \B表示不在单词的边界	\bfood\b \Boo\b
分组	()	描述子模式，表示其作为一个整体	(fo){1,3}
选择		用于选择匹配多个可能的模式（注意不一定是单个字符）中的一个	foo.(org com net)
转义	\	如果其后跟的为非数字和非英文字母，匹配下一个字符。如果其后跟的字符为数字或特殊序列，则表示特殊含义。如果其后跟英文字母，但是非特殊序列，则抛出异常	\\或*或\d或\1

正则表达式元字符：字符类

正则表达式	描述	示例
[set of characters]	匹配中括号里面枚举出的多个字符中的任一字符。第一个字符为^表示否定，-在中间出现表示范围，\表示转义。其他元字符都失去其含义，表示字符本身	[abc] [[abc] [-abc^\n\]]
-	[]内部使用，描述指定范围的字符。如果出现在最初或最后，匹配连字符本身	[0-9a-fA-F] [a-] [-ab]
^[set of characters]	否定枚举，匹配不在后面描述的字符集中的字符。^出现在后面时匹配^本身	[^xyz] [^a-zA-Z]

- **[abc]**: 匹配a或者b或者c
 - **[[abc]** 匹配左中括号或者a或者b或者c
 - **[-abc^\n\]** 匹配连字符，或者a或者b或者c，或者插入符号^，或者换行符，或者右中括号
- **[a-z]**: 匹配小写英文字母
 - **[a-]** 匹配a或连字符
 - **[-ab]** 匹配连字符或者a或者b
- **[^xyz]**: 匹配除了xyz外的其他字符
 - **[^a-zA-Z]** 匹配非英文字母

```
# ]出现在第1个位置，^不出现在第1个位置
# -出现在第1或最后一个位置
>>> re.findall(r'[]^[-]', '[^abc0-9]')
['[', '^', '-', '']
```

```
>>> text = '10 + 45 = 55; 0x2f - 0x1A = 21'
>>> re.findall(r'0[Xx][0-9a-fA-F][0-9a-fA-F]', text)
['0x2f', '0x1A']
```

```
>>> re.findall(r'^ 0-9=+-]', text)
[';', 'x', 'f', 'x', 'A']
```

正则表达式元字符：预定义字符类

预定义字符类：比较常用的字符类，可以在字符类[...]内使用，也可出现在字符类以外

- **匹配选项缺省隐含re.U**，表示采用unicode意义上的数字、字母等的定义
- 选项 re.A(re.ASCII) 表示采用ASCII字符集中的数字、字母等定义

正则表达式	说明
\d	匹配(十进制)数字类字符(isdecimal)。flags=re.A时，等价于[0-9]
\D	与\d含义相反，匹配非数字类字符，相当于[^\d]
\w	匹配单词类字符(字母数字下划线)。flags=re.A时，相当于[a-zA-Z0-9_]
\W	与\w含义相反, 匹配非单词类字符，相当于[^\w]
\s	匹配空格类字符(isspace)。flags=re.A时，相当于[\t\n\r\f\v]
\S	与\s含义相反，匹配非空格类字符，相当于[^\s]
.	匹配除换行符以外的任意单个字符。flags=re.S(re.DOTALL)时，也匹配换行符

```
>>> re.findall(r'.', 'ab\ncd')
['a', 'b', 'c', 'd']
>>> re.findall(r'.', 'ab\ncd', re.S)
['a', 'b', '\n', 'c', 'd']
```

```
>>> text = '10 + 45 = 55; 0x2f - 0x1A = 21'
>>> re.findall(r'0[Xx][\da-fA-F][\da-fA-F]', text)
['0x2f', '0x1A']
```

```
>>> text = 'One1 Two_2\tThree+3\n程序设计\n'
>>> re.findall(r'\w\w\w\s', text)
['ne1 ', 'o_2\t', '序设计\n']
>>> re.findall(r'\w\w\w\s', text, re.A)
['ne1 ', 'o_2\t']
>>> re.findall(r'\w\w\w\S', text)
['One1', 'Two_', 'Thre', '程序设计']
```

正则表达式元字符：转义字符 \

转义字符\，具体含义要检查之后的字符

- \后跟着**非数字和非英文字母的字符**，表示**匹配该字符(前面的\去除)**。如\\ 匹配反斜杠本身 \. 匹配dot \\s匹配\s
- \后跟**整数N(1到99)**，表示匹配前面**编号为N的子模式匹配的内容**
- \后跟着英文字母字符：
 - **预定义字符类**： \d \D \s \S \w \W
 - **边界匹配**： \A \Z \b \B
 - 注意字符串字面量定义中\b表示Backspace字符。但在正则表达式中\b表示单词的开始或结束位置
 - **字符串字面量定义类似的转义序列**： \a \t \r \n \v \f
 - \xHH 2个十六进制数字， \OOO 3个八进制数字，表示相应ASCII码对应的字符
 - \uXXXX 或 \UXXXXXXXX 4位或8位十六进制的Unicode码对应的字符
 - \N{Unicode字符名} 指定名字的字符
 - \后跟英文字母字符，且不在上面给出的合法序列中，则会**抛出异常re.error(bad escape)**

```
>>> re.findall(r'\kabc', 'test')
...
re.error: bad escape \k at position 0
```

```
>>> text = 'sample text: [π = 3.14] [π = 3,14] '
>>> re.findall(r'[π = 3.14]', text)
[' ', ' ', 'π', ' ', '=', ' ', '3', '.', '1',
 '4', ' ', 'π', ' ', '=', ' ', '3', '1', '4']
>>> re.findall(r'\[π = 3.14\\]', text)
['[π = 3.14]', '[π = 3,14]']
```

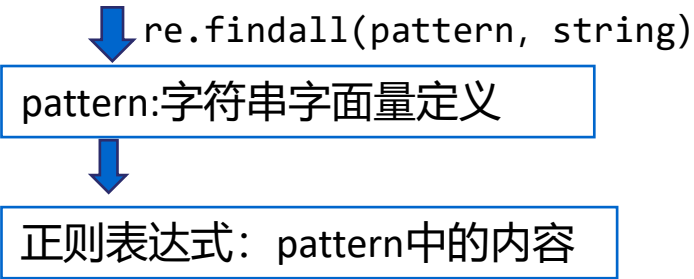
问题：试图找到 [π = 3.14]
正则表达式应该为 \[π = 3\\.14\\]

```
>>> re.findall(r'\[π = 3\\.14\\]', text)
['[π = 3.14]']
>>> re.findall('\\[π = 3\\.14\\]', text)
['[π = 3.14]']
```

正则表达式元字符：转义字符 \

要匹配的模式建议采用原始字符串字面量定义

- 使用原始字符串字面量定义字符串时无需考虑转义
- 要匹配\t \n等字符时也可以采用原始字符串定义！
 - 正则表达式支持字符串字面量定义中的类似转义序列：比如模式r'\n' 匹配换行符
 - 特例：要匹配\b（删除字符）时，不能采用r'\b'。re模块解释为单词的边界
 - r'\x08'可匹配\b(删除字符)，\b的ASCII码为8
 - r'[\b]'也可匹配\b(删除字符)



问题：试图找到 `[π = 3.14]`
正则表达式应该为 `\[π = 3\.14\]`

```
>>> re.findall(r'\[π = 3\.14\]', text)
['[π = 3.14]']
>>> re.findall('\[π = 3\.14\]', text)
['[π = 3.14]']
```

```
>>> text = r'\s match any whitespace character'
>>> re.findall(r'\s', text)
[' ', ' ', ' ', ' ']
>>> re.findall(r'\\s', text)
['\\s']
>>> re.findall('\\\\s', text)
['\\\\s']
```

正则表达式	含义	原始字符串定义	字符串定义
\s	空格类字符	r'\s',	'\\s'
\\s	\s	r'\\s'	'\\\\s'

正则表达式元字符：转义字符 \

正则表达式	描述	示例
[set of characters]	匹配中括号里面枚举出的多个字符中的任一字符。第一个字符为^表示否定，-在中间出现表示范围，\表示转义。其他元字符都失去其含义，表示字符本身	[abc] [[abc] [-abc^\n\]]
-	[]内部使用，描述指定范围的字符。如果出现在最初或最后，匹配连字符本身	[0-9a-fA-F] [a-] [-ab]
^[set of characters]	否定枚举，匹配不在后面描述的字符集中的字符。^出现在后面时匹配^本身	[^xyz] [^a-zA-Z]

字符类[...]支持转义，支持字符串字面量类似的转义方式和预定义字符类。r'[\u4E00-\u9FA5]' 匹配汉字

- 支持\t \n \r \v \f \a \b等
- \xHH 2个十六进制数字，\OOO 3个八进制数字，\uXXXX 或 \UXXXXXXXX \N{Unicode字符名} 指定名字的字符
- 预定义字符类：\d \D \s \S \w \W
- \后面为非英文和非数字字符时，表示该字符本身

```
>>> text = 'sample text: [π = 3.14] [π = 3,14]'\n>>> re.findall(r'3\.14', text)\n['3.14']\n>>> re.findall(r'3[.]14', text)\n['3.14']\n>>> re.findall(r'[\u03B1-\u03C9]', text) # 小写希腊字母\n['π', 'π']\n>>> re.findall(r'[\u4E00-\u9FA5]', 'python程序设计') # 汉字\n['程', '序', '设', '计']
```

[.]等价于\.

```
>>> re.findall(r'^[-]', '^abc0-9')\n['^', '^', '^', '^']\n>>> re.findall(r'\\^\\[-]', '^abc0-9')\n['^', '^', '^', '^']
```

```
special = '^[-]'\nre.findall('[' + re.escape(special) + ']',\n           '^abc0-9')
```

re.escape(pattern)

- 返回字符串，对于pattern中那些具有特殊含义的字符，会在其前面插入一个反斜杠，这样其作为正则表达式模式时**匹配pattern本身**
- 可动态**构建一个正好匹配pattern本身**的正则表达式，避免pattern里面的字符被re解释为特殊含义

```
>>> print(re.escape(r'\s'))  
\\s  
>>> print(re.escape(r'http://www.python.org'))  
http://www\.python\.org  
>>> print(re.escape('^[-]'))  
\\^[\\-\\]
```

```
def re_escape():  
    text = """regexp = re.compile('((jump)ed for) (food)')  
    text='The quick brown fox jumped for food. The lazy black dog jumped for food.'  
    print(regexp.findall(text))  
    """  
  
    pattern = input("请输入要匹配的内容:")  
    esc_pattern = re.escape(pattern)  
    print('正则表达式为', esc_pattern)  
    matches = re.findall(esc_pattern, text)  
    print(matches)
```

escape.py

正则表达式元字符：边界匹配符

0宽度边界匹配符：在**指定的位置**出现时匹配，其匹配的结果为空字符串，**不会消耗**文本中的字符

元字符	说明	示例
^	匹配字符串的 开始位置 。flags包括re.M或re.MULTILINE，即多行模式时，匹配每行行首(字符串开始或\n的下一位置)	^a 匹配 a ababa的第1个a
\$	匹配 最后一行的行尾 (即字符串结尾，或字符串最后位置为换行符时还包括最后那个换行符之前的位置)。多行模式下匹配每行的行尾(\n之前的位置)以及字符串结尾	a\$ 匹配ababa a 中的最后的a
\A \Z	匹配字符串开始的位置或字符串结束位置	
\b \B	匹配单词的边界位置(单词开始或结尾)，或者匹配非单词的边界位置	

```
>>> text = ""# 1
# 2

\t# 1234

# 4
""
>>> print(repr(text))
'# 1\n# 2\n\n\t# 1234\n\n# 4\n'
```

```
>>> re.findall('^#', text)
['#']
>>> re.findall('^#', text, re.M)
['#', '#', '#']
>>> re.findall(r'\d$', text)
['4']
>>> re.findall(r'\d$', text, re.M)
['1', '2', '4', '4']

>>> re.findall(r'^$', text, re.M)
['', '', '']
```

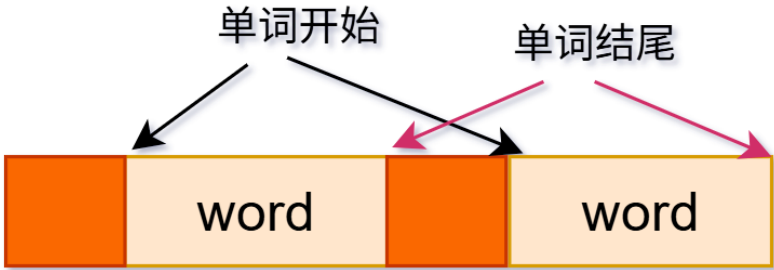
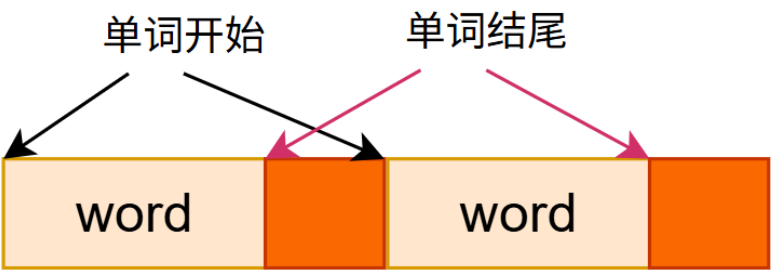
Bug?

- ^和\$都可以匹配空字符串中的开始和结束位置

正则表达式元字符：边界匹配符

o宽度边界匹配符：在指定的位置出现时**匹配**，其匹配的结果为**空字符串**，**不会消耗**文本中的字符

元字符	说明	示例
\b	匹配单词边界（即单词开始或单词结尾）	\bhi\b 匹配hi (hi) him hi4 hi中的前面2个hi和最后一个hi
\B	与\b含义相反，表示不在单词的边界位置	- py\B 匹配python py3 happy py!中的前面两个py \B@匹配 @dlmao (@dlmao) dlmao@fudan中的前面两个@



- 单词开始：前面为非单词类字符或字符串开始，当前位置为单词字符
- 单词结尾：当前为单词字符，后面为非单词字符或字符串结尾。
- 非单词边界：除了单词边界外的其他位置
 - 当前为单词字符，前面为单词字符
 - 当前为非单词字符，前面为字符串开始或非单词字符
 - 当前为字符串结尾，前面为非单词字符

正则表达式元字符：边界匹配符

o宽度边界匹配符：在指定的位置出现时**匹配**，其**匹配的结果为空字符串**，**不会消耗**文本中的字符

元字符	说明	示例
\b	匹配单词边界（即单词开始或单词结尾）	\bhi\b 匹配hi (hi) him hi4 hi中的前面2个hi和最后一个hi
\B	与\b含义相反，表示不在单词的边界位置	- py\B 匹配python py3 happy py!中的前面两个py \B@匹配 @dlmao (@dlmao) dlmao@fudan中的前面两个@

```
>>> text = "foo foo.(foo)bar baz foobar foo3 4foo [4foo4] barfoo 'foo' _foo foo"
>>> re.findall(r'foo', text)
['foo', 'foo', 'foo', 'foo', 'foo', 'foo', 'foo', 'foo', 'foo', 'foo', 'foo']
>>> re.findall(r'\bfoo\b', text)      # 匹配单词foo
['foo', 'foo', 'foo', 'foo', 'foo']
>>> re.findall(r'\bfoo\b', text)      # \b表示退格字符
[]
>>> re.findall(r'\Bfoo\b', text)      # 单词以foo结尾时，匹配该foo
['foo', 'foo', 'foo']
>>> re.findall(r'\bfoo\B', text)      # 单词以foo开始时，匹配该foo
['foo', 'foo']
>>> re.findall(r'\Bfoo\B', text)      # 单词中间包含foo时，匹配该foo
['foo']
```

正则表达式元字符：重复限定符

x重复多次。x可为单个字符（包括字符类等）或通过小括号包含的子模式（作为一个整体）

元字符	说明	示例
X{m,n}	X重复m到n次	o{1,3} 匹配f ooo o oo d中的前3个"o"和后3个"o"
X{m,}	X至少重复m次	o{2,} 匹配f oooooo d中的所有o, 不匹配bob中的o
X{,n}	X重复0到n次, 等价于X{0,n}	0{,2}1 匹配 1, 01, 001
X{m}	X重复m次	[0-9]{3} 匹配 000 ~ 999
X+	X重复1次或多次, 等价于X{1,}	zo+ 匹配zo, zoo, 但不匹配z
X*	X重复0次或多次, 等价于X{0,}	zo* 匹配zo, zoo, z
X?	X可选, 即X重复0或1次, 等价于X{0,1}	colou?r 匹配color, colour



- 默认采用**贪婪(greedy)匹配算法**：正则表达式引擎**尽可能多**匹配要重复的前导字符或子模式，只有当这种重复引起整个模式匹配失败的情况下，引擎才会进行回溯（尝试稍短的匹配）

```
>>> text = 'The quick brown fox jumped for food.'
>>> re.findall(r'\b\w+\b', text) # r'\w+'
['The', 'quick', 'brown', 'fox', 'jumped', 'for', 'food']
>>> re.findall(r'\b\w+[a-e]\b', text)
['The', 'jumped', 'food']
```

课堂思考

- 匹配“空行”：该行没有包含任何字符或者仅仅包含空格类字符
- 匹配注释行：包含了#，并且#出现在行首，或者之前有多个空格

正则表达式元字符之重复限定符：懒惰匹配算法

如果在重复限定符后面加后缀"?", 表示使用**懒惰(lazy match)匹配算法**

- 贪婪匹配算法**是指正则表达式引擎**尽可能多**匹配要重复的前导字符或子模式，只有当这种重复引起整个模式匹配失败的情况下，引擎才会进行回溯（尝试稍短的匹配）
- 懒惰匹配算法**是指正则表达式引擎**尽可能少**地进行重复匹配，只有当这种重复引起整个正则表达式**匹配失败**的情况下，引擎会进行回溯(匹配更多的字符)

符号	说明
X{m,n}?	重复m到n次，但尽可能少重复
X{m,}?	重复m次以上，但尽可能少重复
X{,n}?	重复0到n次，但尽可能少重复
X+?	重复1次或更多次，但尽可能少重复
X*?	重复任意次，但尽可能少重复
X??	重复0次或1次，但尽可能少重复

part: C not in part

C

```
>>> text = '<book><title>Python</title></book>'
>>> re.findall('<.+>',text)
['<book><title>Python</title></book>']
>>> re.findall('<[^>]+>', text)
['<book>', '<title>', '</title>', '</book>']

>>> re.findall(r'<.+?>', text)
['<book>', '<title>', '</title>', '</book>']
```

.+?>	尽可能少地匹配一个以上字符，后面为>
[^>]+>	尽可能多地匹配一个以上非>字符，后面为>

.+?C

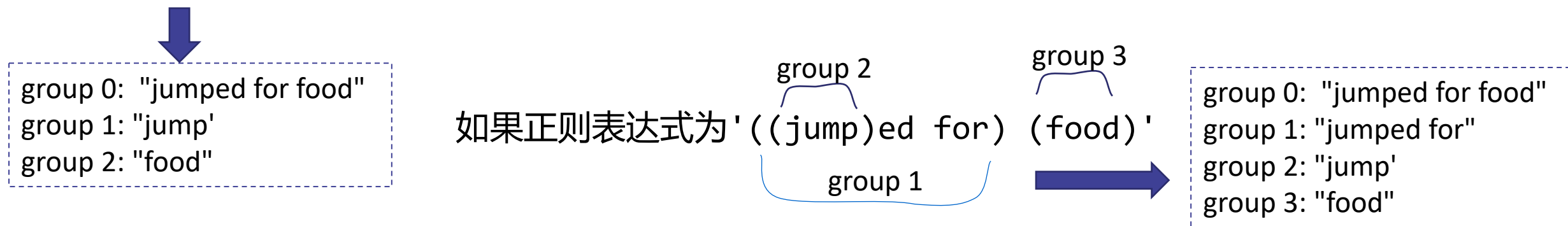
[^C]+C

正则表达式元字符：分组符(group)

要匹配的模式中，可以将其分成多个部分，其中的几个特定的部分加上括号，这些加括号的部分 **(pattern)** 称为分组(group)或子模式

- ① 在获得模式所匹配的内容的同时也可以获得各个（捕获）子模式匹配的内容
 - ② **()** 包含的子模式内的内容作为一个整体出现，可以应用前面介绍的重复限定符，表示 **子模式重复多次**。比如 `(pattern){2}` 表示前面的内容与 `pattern` 匹配，之后的内容也与 `pattern` 匹配
 - ③ 支持 **反向引用(backreference)**，匹配前面相应(捕获)子模式所匹配的内容
- 子模式可以嵌套
 - (捕获)子模式有一个数字编号：按照从左到右的 **左括号的顺序** 从 **1开始** 进行编号。 **编号0** 表示整个模式

'(jump)ed for (food)' 匹配字符串 'The quick brown fox jumped for food'



正则表达式元字符：子模式之findall

① 在获得模式所匹配的内容的同时也可以获得各个（捕获）子模式匹配的内容

`re.findall(pattern, string, flags=0)` `regex.findall(string[, pos[, endpos]])`

- 在字符串string中搜索与模式pattern匹配的内容，接下来继续搜索下一个匹配...，找到的匹配组成一个**列表**返回。flags给出了搜索时采用的匹配选项
- 如果模式中没有**组(子模式)**，则列表的每个元素为与模式匹配的内容（字符串）
- 如果模式中有**组(子模式)**，则匹配结果中仅包括与子模式匹配的内容，整个模式匹配的内容(group 0)并没有包含在内。如果有**多个组(子模式)**，则每次匹配的结果以tuple形式组织，即[(**group 1,group2...,groupN**), (第2个匹配), ..., (最后一次匹配)]
- 如果最后输出既想要子模式也想要模式匹配的内容，则引入一个包含了整个模式的group

```
def re_findall():  
    text='The quick brown fox jumped for food. The lazy black dog jumped for food.'  
    print(re.findall('jumped for food', text)) #没有group, 包含匹配的模式  
    print(re.findall('jumped for (food)', text))  
    print(re.findall('((jump)ed for) (food)', text)) #4个group, group 1包含所有内容
```

group.py

['jumped for food', 'jumped for food']

['food', 'food']

[('jumped for food', 'jumped for', 'jump', 'food'), ('jumped for food', 'jumped for', 'jump', 'food')]

正则表达式元字符：子模式重复

② **(pattern)** 包含的子模式内的内容作为一个整体出现,可以应用重复限定符, 表示**子模式重复多次**

- 子模式重复多次时, 在最后输出的那个分组中的内容是其**最后一次重复**所匹配的内容
- 如果子模式重复允许0次, 而且**匹配的正好是0次**, 则findall中输出的那个分组内容为**空字符串**

重复多次



IP地址: 202.120.225.10

4个整数以dot分隔, 每个整数包括1到3个十进制数字

- (ddd.)重复3次 + ddd

```
>>> text = '202.120.225.10 8.8.8.8 127.0.0.1'
>>> re.findall(r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}', text)
['202.120.225.10', '8.8.8.8', '127.0.0.1']
>>> re.findall(r'(\d{1,3}\.){3}\d{1,3}', text)
['225.', '8.', '0.']
>>> re.findall(r'((\d{1,3}\.){3}\d{1,3})', text)
[('202.120.225.10', '225. '), ('8.8.8.8', '8. '), ('127.0.0.1', '0. ')]
```

匹配: 单词+单词前面的空格类字符 (如果有)

```
>>> re.findall(r'(\s)*(\w+)', 'hello world')
[('', 'hello'), (' ', 'world')]
```

正则表达式	说明
(pattern)?	允许模式重复0次或1次
(pattern)*	允许模式重复0次或多次
(pattern)+	允许模式重复1次或多次
(pattern){m,n}	允许模式重复m~n次
(pattern){m,}	允许模式重复至少m次
(pattern){,n}	允许模式重复0~n次
(pattern){m}	允许模式正好重复m次

- 缺省贪婪匹配, 后面加?表示懒惰匹配. 比如(pattern)+?

③ 支持**反向引用(backreference)**，匹配前面相应(捕获) 子模式所匹配的内容

正则表达式	描述
<code>\N</code>	匹配前面编号为N的分组所匹配的内容。N不能是后面的分组编号，也不能为0

寻找连续出现的单词

```
>>> text = "Paris in the the spring"
>>> re.findall(r'(\b\w+)\s+\1', text)
['the']
```

寻找重复数：由多个相同数字组成的数

```
>>> text = "1 22 3333"
>>> results = re.findall(r'\b((\d)\2*)\b', text)
>>> results
[('1', '1'), ('22', '2'), ('3333', '3')]
>>> [x[0] for x in results]
['1', '22', '3333']
```

`\b((\d)\2*)\b`

- group2: 匹配数值，比如3
- `\2*` group 2的匹配重复0到多次，即3重复0到多次

正则表达式元字符：选择符

pattern1|pattern2|pattern3

- 选择符"**|**", 用于选择匹配多个可能的子模式 (注意不一定是单个字符) 中的一个
- 按照从左到右的顺序匹配, 即首先尝试匹配pattern1, 如果不匹配, 尝试pattern2
- 如需要, 可使用**()**来限制选择符的作用范围(pattern1|pattern2|pattern3)

匹配bat或cat开头的单词

```
>>> re.findall(r'(\b(bat|cat)\w+)', 'batch bathroom catch catbird')
[('batch', 'bat'), ('bathroom', 'bat'), ('catch', 'cat'), ('catbird', 'cat')]
>>> re.findall(r'(\b(b|c)at\w+)', 'batch bathroom catch catbird')
[('batch', 'b'), ('bathroom', 'b'), ('catch', 'c'), ('catbird', 'c')]
# (bat|cat) --> [bc]at
>>> re.findall(r'\b[bc]at\w+', 'batch bathroom catch catbird')
['batch', 'bathroom', 'catch', 'catbird']
```

主要内容

- 正则表达式
- **模式对象和相应方法**
- 子模式扩展(可选的内容)

re模块的方法与模式对象的方法

re.findall(pattern, string, flags) 等价 re.compile(pattern, flags).findall(string)

- re模块的搜索类的方法**无法传递pos/endpos参数**，即搜索的范围只能为整个字符串

re.Pattern模式对象的方法	re模块的方法
编译正则表达式，返回类型为re.Pattern的对象	re.compile(pattern, flags=0)
regex.search(string[, pos[, endpos]])	re.search(pattern, string, flags=0)
regex.match(string[, pos[, endpos]])	re.match(pattern, string, flags=0)
regex.fullmatch(string[, pos[, endpos]])	re.fullmatch(pattern, string, flags=0)
regex.finditer(string[, pos[, endpos]])	re.finditer(pattern, string, flags=0)
regex.findall(string[, pos[, endpos]])	re.findall(pattern, string, flags=0)
regex.sub(repl,string[,count=0])	re.sub(pattern, repl, string, count=0, flags=0)
regex.subn(repl,string[,count=0])	re.subn(pattern, repl, string, count=0, flags=0)
regex.split(string[, maxsplit = 0])	re.split(pattern, string, maxsplit=0, flags=0)
返回字符串，该字符串对于pattern中有特殊含义的正则表达式元字符进行转义	re.escape(pattern) 比如print(re.escape(r'\\')) # 输出 \\ \\

search方法

```
regexp = re.compile(pattern, flags=0); regexp.search(string[, pos[, endpos]])
```

```
re.search(pattern, string, flags=0)
```

- 在字符串string或其指定范围string[pos:endpos] 搜索第一个与模式匹配的内容。若找到，返回一个描述匹配信息的**Match对象(真值判断为真)**，否则返回None。Match对象类型为re.Match，可获得匹配位置和内容

Match对象方法	说明
group([group1,...])	返回匹配的1个或多个 捕获子模式 (编号或者名字) 所对应的内容。如果不传递参数，表示group 0，即模式对应的内容。如果传递多个参数，则返回元组，每个元素为相应子模式匹配的内容。比如group(1,2)返回group 1和group 2的匹配内容组成的元组。group('city')返回名字为city的group匹配的内容
start(group=0)	返回相应子模式所匹配内容在原字符串的起始位置的下标。group默认为组0(整个模式)
end(group=0)	返回相应子模式所匹配内容在原字符串的结束位置(开区间，即匹配内容的下一个位置)
span(group=0)	返回相应子模式所匹配内容在原字符串的开始和结束位置的下标，即(m.start(), m.end())
groups()	返回一个包含匹配的所有捕获的子模式(1,2,...)的内容的元组，注意不包括group 0
groupdict()	返回包含匹配的所有捕获的 命名子模式 内容的字典，key=子模式名, value为匹配的内容
属性re, string	返回进行匹配时采用的模式对象regexp，搜索的字符串string

search方法示例: `regex.search(string[, pos[, endpos]])`

```
>>> import re
>>> regexp = re.compile('((jump)ed for) (food)')
>>> text = 'The quick brown fox jumped for food.
The lazy black dog jumped for food.'
>>> match = regexp.search(text)
>>> match
<_sre.SRE_Match object; span=(20, 35),
match='jumped for food'>
>>> match.group()
'jumped for food'
>>> match.group(1, 2)
('jumped for', 'jump')
>>> match.groups()
('jumped for', 'jump', 'food')
>>> match.start()
20
>>> match.end()
35
>>> match.span()
(20, 35)
```

```
>>> match.start(1)
20
>>> match.end(1)
30
>>> match.span(1)
(20, 30)
>>> match2 = regexp.search(text, 35)
>>> match2
<_sre.SRE_Match object; span=(56, 71),
match='jumped for food'>
>>> match3 = regexp.search(text, 71)
>>> print(match3)
None
```

```
pos = 0      # 从头开始搜索 (pos=0)
while True:
    match = regexp.search(text, pos)
    if not match:
        break
    pos = match.end()
```

search方法示例

1. 编译模式对象: `regexp=re.compile(pattern)`
2. 下次搜索开始的位置`pos = 0`
3. 重复直到调用 `regexp.search(text,pos)`返回match对象为空 (即没有更多的匹配)
 - 从match对象中获得找到的匹配以及相应位置
 - 更新`pos = match.end()`

group.py

```
import re
regexp = re.compile('((jump)ed for) (food)')
text='The quick brown fox jumped for food. The lazy black dog jumped for food.'
pos = count = 0      # 从头开始搜索 (pos=0)
while True:
    match = regexp.search(text, pos)      #从上次匹配的位置之后继续搜索模式
    if not match:      # 找不到, 退出循环
        break
    count += 1      # 第几次找到?
    print('{} found "{}" at [{} , {}]'.format(count, match.group(), *match.span()))
    for k, v in enumerate(match.groups(), 1):
        print('group %d: %s, span %s' % (k, v, match.span(k)))
    pos = match.end()      # 记录上次匹配的内容之后所对应的位置, 即下一次搜索开始的位置
```

finditer 方法

- search方法只能找到第一个与模式匹配的match对象
- finditer(string[, pos[, endpos]]): 返回一个迭代器, 每次返回匹配的下一个match对象

group.py

```
def re_finditer():  
    regexp = re.compile('((jump)ed for) (food)')  
    text = 'The quick brown fox jumped for food. The lazy black dog jumped for food.'  
    matches_iter = regexp.finditer(text)  
    for count, match in enumerate(matches_iter, 1):  
        print('{} found "{}" at [{} , {}]'.format(count, match.group(), *match.span()))  
        for k, v in enumerate(match.groups(), 1):  
            print('    group %d: %s, span %s' % (k, v, match.span(k)))
```

判断某个字符串是否以指定的模式开始，或者某个字符串是否与某个模式完全匹配：

- `match(string[, pos[, endpos]])`与`search`类似，只是要求模式(pattern)出现在**字符串开头**或指定范围的开头。只有一个参数时，类似于`^pattern`
- `fullmatch(string[, pos[, endpos]])` 要求**模式完整匹配**字符串，即模式的最前面为字符串开头或指定范围的开头，最后面为字符串尾部或指定范围的尾部。只有一个参数时类似于`^pattern$`

```
>>> import re
>>> text='The quick brown fox jumped for food. The lazy black dog jumped for food.'
>>> regexp = re.compile('((jump)ed for) (food)')
>>> match = regexp.match(text)      # text以jumped for food开始时匹配
>>> print(match)
None
>>> regexp.match(text, 20)          # text[20:]以jumped for food开始时匹配
<re.Match object; span=(20, 35), match='jumped for food'>
>>> regexp2 = re.compile('the', re.I)
>>> match = regexp2.match(text)      # text是否以the开始 (大小写无关)
>>> match
<_sre.SRE_Match object; span=(0, 3), match='The'>
>>> regexp2.fullmatch('The')        # The是否与the匹配 (大小写无关)
<_sre.SRE_Match object; span=(0, 3), match='The'>
```

替换sub和subn

<pre>re.sub(pattern, replace, string, count=0, flags=0) regex.sub(replace, string, count=0)->str</pre>	将字符串string中与pattern匹配的内容用replace替换，返回新的字符串。count表示最多替换多少次，缺省为0，表示全部替换。replace可以是字符串或函数对象
<pre>re.subn(pattern, replace, string, count=0, flags=0) regex.subn(replace, string, count=0)->(str, int)</pre>	与sub类似，只是返回的是一个元组，第1个元素为替换后产生的新字符串，第2个元素为总共替换的次数

replace为字符串时，其中还可以包含分组引用：

- \N或者\g<N> 表示第N(不为0) 个分组匹配的内容
- \g<name> 表示名字为name的分组匹配的内容

```
text = '''Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.'''
```

sub.py

```
# 将以字母"b"和"B"开头的单词替换为"*"
pattern = r'\b[bB]\w*\b'
print(re.sub(pattern, '*', text))

# 将第一个以字母"b"和"B"开头的单词替换为"*"
print(re.sub(pattern, '*', text, 1))

# 将以字母"b"和"B"开头的单词前面添加*
pattern = r'\b([bB]\w*)\b'
print(re.sub(pattern, r'*\1', text))
print(re.sub(pattern, r'*\g<1>', text))
```

替换sub和subn

re.sub(pattern, replace, string, count=0, flags=0)

- 参数replace传递的为字符串，表示替换成replace，在replace中可包含分组引用
- 参数replace也可以传递一个函数对象，该函数的参数为pattern所匹配的match对象。表示替换为调用replace(matchobj)所返回的字符串

sub.py

```
def re_sub_func():  
    pattern = r'\b([BC]\w*)\b\s+is\s+(better|worse)\s+than\s+(\w+)\b'  
    # B或C开头的单词 一个以上空格 is 空格 better或worse 空格 than 空格 单词  
    def repl_func(match):  
        first = match.group(1)  
        last = match.group(3)  
        return '%s is %s than %s' % (last.capitalize(),  
                                     'worse' if match.group(2) == 'better' else 'better', first.lower())  
    text_ = re.sub(pattern, repl_func, text)  
    print(text_)
```

Word1

is

better/worse

than

word2

Word2

is

worse/better

than

word1

split

`re.split(pattern, string, maxsplit=0, flags=0)` `regex.split(string, maxsplit=0)`

- 根据pattern分割字符串string，返回分割后的字符串列表。maxsplit为分割的最大次数，缺省为尽可能最大可能分割
- 拓展：缺省分隔符不会保留，但是如果分隔符pattern中包含捕获子模式，则pattern中所有子模式的内容也在返回的字符串列表中→ 通过`(?:...)`表示该子模式不要捕获(capture)

```
text = 'alpha. beta....gamma delta'
print(re.split(r'[. ]+', text))
print(re.split(r'[. ]+', text, maxsplit=1))
```

分隔符为一个以上字符(可以是.或空格)

```
['alpha', 'beta', 'gamma', 'delta']
['alpha', 'beta...gamma delta']
```

```
>>> re.split(r'(+|;+)', '1,,2;;3')
['1', ',', '2', ';;', '3']
```

```
>>> re.split(r'(?:,+|;+)', '1,,2;;3')
['1', '2', '3']
```

`' \t Hello, world! \n'` → `['Hello,', 'world!']` → `'Hello, world!'`

```
# 实现 text.split()
def my_split(text):
    return re.split(r'\s+', text.strip())

words = re.split(r'\s+', text)
return [word for word in words if word]
```

```
text = ' \t Hello, world! \n'
def squeeze_whitespace(text):
    return re.sub(r'\s+', ' ', text.strip())

return ' '.join(my_split(text))
```

split.py

匹配选项(可选): `re.VERBOSE`

- Pattern和string都是str对象, 选项缺省为`re.U`
- Pattern和string都是bytes对象, 选项缺省为`re.L`

`re.findall(pattern, string, flags=0)` `re.compile(pattern, flags=0)`

- `flags`: 匹配选项。传递的值为0时表示采用缺省的匹配选项 (`re.U`)
- 可以采用短格式(如`re.I`), 也可采用长格式(如`re.IGNORECASE`)
- 多个选项可以通过运算符 `|` (按位或) 组合, 比如 `re.I | re.U`

```
pattern = r"""
    (?P<first_three>[\d]{3}) # The first three digits...
    -                       # A literal hyphen...
    (?P<last_four>[\d]{4})  # The last four digits...
"""
# r'(?P<first_three>[\d]{3})-(?P<last_four>[\d]{4})'
results = re.findall(pattern, '867-5309', re.VERBOSE)
print(results)
```

匹配选项(flags)	说明
<code>re.I</code> <code>re.IGNORECASE</code>	忽略大小写
<code>re.L</code> <code>re.LOCALE</code> 忽略这个选项	<code>\w \W \b \B \s \S</code> 与本地字符集有关, <code>re</code> 模块用于 <code>bytes</code> 对象时使用
<code>re.U</code> <code>re.UNICODE</code>	<code>\w \W \b \B \s \S</code> 的定义基于Unicode, <code>pattern</code> 为str时缺省选项
<code>re.A</code> <code>re.ASCII</code>	<code>\w \W \b \B \s \S</code> 的定义基于ASCII
<code>re.S</code> <code>re.DOTALL</code>	元字符 <code>.</code> (dot) 也匹配换行符
<code>re.X</code> <code>re.VERBOSE</code>	忽略模式中的空格类字符 (这样可以写成多行, 也可添加空格), 并可以使用 <code>#</code> 注释, 提高可读性
<code>re.M</code> <code>re.MULTILINE</code>	多行匹配模式, <code>^</code> <code>\$</code> 可以匹配每行的开始和结尾

示例：子模式

示例11-2变种1：（1）获取文本中的手机号码（11位数字：第1个数字为1，第2个数字为3到9）

（2）获取文本中的固定电话，区号-电话号码，返回列表，每个元素为(区号, 电话号码)。

文本中的区号可选（默认为021），3到4位，第1位为0，电话号码7-8位

```
text = r"""Fudan: 021-65642222
zju: 0571-87951111
tel1: 65643333
MP: 13988888888
MP: 13888888888
"""
```

某个group没有匹配，包括group重复多次时其重复次数为0

- findall(.)：其匹配的结果为空字符串
- match对象：其匹配的结果为None

```
mobile_phone_re = r'\b1[3-9]\d{9}\b'
phone_re = r'\b((0\d{2,3})-)?(\d{7,8})\b'
mobile_phones = re.findall(mobile_phone_re, text)
print('Mobile Phones:', ', '.join(mobile_phones))
# Mobile Phones: 13988888888, 13888888888
phones = re.findall(phone_re, text)
phones = [(area or '021', local) for _, area, local in phones]
print('Phones:', ', '.join([f'{area}-{local}' for area, local in phones]))
# Phones: 021-65642222, 0571-87951111, 021-65643333
phones = []
for match in re.finditer(phone_re, text):
    phones.append((match.group(2) or '021', match.group(3)))
```

parse_phone.py

示例：选择符

案例2：日志文件中错误信息的提取

- 提取所有以 “Error:” 或 “[Error]” 标记的错误信息行

parse_log.py

```
import re
log = """... # 日志文件的文本
[Error] Connection lost
Warn: System warning
Error: Invalid user input
Info: User Exist
..."""
error_pattern = r"Error:.\+|\[Error\].\+"
# error_pattern = r"(Error:|\[Error\]).\+"
errors = re.findall(error_pattern, log)
for error in errors:
    print(error)
```

[Error] Connection lost
Error: Invalid user input

示例：懒惰匹配

- 案例3：从网页代码中提取链接 【代码11-9】： href="...."

```
import re
html = """<html>
<head></head>
<body>
<a href="http://example.com/1">Link 1</a>
<a href="http://example.com/2">Link 2</a>
<a href="http://example.com/3">Link 3</a>
</body>
</html>"""
links = re.findall(r'href="(.*?)"', html)
for link in links:
    print(link)
```

parse_hyperlinks.py

```
http://example.com/1
http://example.com/2
http://example.com/3
```

示例：fullmatch

- 案例4：验证用户输入的格式

validate_phone.py

```
import re
phone_number = "13812345678"
pattern = r'^1[3-9]\d{9}$'
if re.search(pattern, phone_number):
    print("有效的电话号码")
else:
    print("无效的电话号码")

pattern = r'1[3-9]\d{9}'
if re.fullmatch(pattern, phone_number):
    print("有效的电话号码")
else:
    print("无效的电话号码")
```

示例：sub

- 案例1：复旦大学的邮箱后缀更新 21xx@fudan.edu.cn替换为 21xx@m.fudan.edu.cn

```
import re
# 示例邮箱列表
emails = [
    "2114student1@fudan.edu.cn",
    "2243student2@fudan.edu.cn",
    "2079student3@fudan.edu.cn"
]
# 正则表达式模式
pattern = r'(2[1-9].*@)fudan\.edu\.cn'
# 替换旧后缀为新后缀
updated_emails = [re.sub(pattern, r'\1m.fudan.edu.cn', email) for email in emails]

for email in updated_emails:
    print(email)
```

update_mailbox.py

2114student1@m.fudan.edu.cn
2243student2@m.fudan.edu.cn
2079student3@m.fudan.edu.cn

示例：sub

- 案例5：批量重命名文件，年-月 ---> 月-年

```
files = ["2020-01-report.txt", "2021-03-summary.txt", "2022-07-feedback.txt"]

for file in files:
    new_name = re.sub(r'(\d{4})-(\d{2})-(.*).txt', r'\2-\1-\3.txt', file)
    print(f'rename {file} --> {new_name}')
    # os.rename(file, new_name) # 实际重命名文件
```

batch_rename.py

rename 2020-01-report.txt --> 01-2020-report.txt

rename 2021-03-summary.txt --> 03-2021-summary.txt

rename 2022-07-feedback.txt --> 07-2022-feedback.txt

主要内容

- 正则表达式
- 模式对象和相应方法
- **子模式扩展（可选内容）**

分组(子模式): 不捕获分组(?:pattern)

- 有的时候可能要进行分组, 但用户并不关心某个子模式具体匹配的内容是什么
- 可以通过在(后面添加?: 表示对该分组匹配的内容不感兴趣, **不捕获(capture)**, 即(?:pattern)
- 不捕获分组也**不占用分组编号**: 意味着无法获取其匹配的内容

示例11-2变种 (2) 获取文本中的固定电话, 区号-电话号码, 返回列表, 每个元素为(区号, 电话号码)。

文本中的区号可选 (默认为021), 3到4位, 第1位为0, 电话号码7-8位

```
r'\b((0\d{2,3})-)?(\d{7,8})\b' # group 1: 区号-, group 2: 区号, group 3: 电话号码
```

```
r'\b(?:0\d{2,3})-)?(\d{7,8})\b' # group 1: 区号, group 2: 电话号码
```

```
text = r"Fudan: 021-65642222 zju: 0571-87951111 tel1: 65643333 ""
phone_re = r'\b(?:0\d{2,3})-)?(\d{7,8})\b'
phones = re.findall(phone_re, text)
phones = [(area or '021', local) for area, local in phones]
print('Phones:', ', '.join([f'{area}-{local}' for area, local in phones]))

phones = []
for match in re.finditer(phone_re, text):
    phones.append((match.group(1) or '021', match.group(2)))
```

parse_phone.py

分组(子模式):命名(?P<name>pattern) (?P=name)

- 给**分组(子模式)命名**, 方便引用, 而且更改正则表达式也不会有影响
- 如果匹配时, match对象:
 - 可以通过分组编号或者分组名访问匹配的内容: group(1), group('area')
 - groupdict(): 返回字典对象, key=分组名, value=匹配的内容

```
r'\b(?: (?P<area>\d{2,3})-)? (?P<local>\d{7,8})\b'
```

```
# group 1(name=area): 区号, group 2(name=local): 电话号码
```

```
text = r"Fudan: 021-65642222 zju: 0571-87951111 tel1: 65643333 ""
phone_re = r'\b(?: (?P<area>\d{2,3})-)? (?P<local>\d{7,8})\b'

phones = []
for match in re.finditer(phone_re, text):
    print(match.groupdict())
    # {'area': '021', 'local': '65642222'}
    phones.append((match.group('area') or '021', match.group('local')))
```

parse_phone.py

```
{'area': '021', 'local': '65642222'}
{'area': '0571', 'local': '87951111'}
{'area': None, 'local': '65643333'}
```

正则表达式元字符：子模式，命名(**?P<name>pattern**) (**?P=name**)

③ 支持**反向引用(backreference)**，匹配前面相应(捕获) 子模式所匹配的内容

正则表达式	描述
<code>\N</code>	匹配前面编号为N的分组所匹配的内容。N不能是后面的分组编号，也不能为0
<code>(?P=name)</code>	匹配前面名字为name的分组所匹配的内容

寻找连续出现的单词

```
>>> text = "Paris in the the spring"
>>> re.findall(r'(\b\w+)\s+\1', text)
['the']
```

寻找连续出现的单词

```
>>> text = "Paris in the the spring"
>>> re.findall(r'(?P<word>\b\w+)\s+(?P=word)', text)
['the']
```

`re.sub(pattern, replace, string, flags=0)` `regex.sub(replace, string)`

替代字符串`replace`中出现:

- **\g<name>** 表示名字为name的分组匹配的内容

分组(子模式):肯定/否定前瞻

- 在匹配时要检查**当前位置之后是否满足或者不满足某个模式**。在进行前瞻性的检查时并不会消耗任何字符,即**零宽度匹配**
- 不引入捕获组**(capturing group): 不会被捕获,不会占用分组编号

(?=pattern)	零宽度肯定前瞻断言(zero-width positive lookahead assertion), 检查当前位置之后的内容, 要求其满足模式pattern, 在检查时采用零宽度匹配
(?!pattern)	零宽度否定前瞻断言(zero-width negative lookahead assertion), 检查当前位置之后的内容, 要求其不与模式pattern匹配, 在检查时采用零宽度匹配

匹配那些后面跟着逗号的单词

```
>>> text = 'They were three: Felix, Victor, and Carlos.'
>>> re.findall(r'\w+', text)
['Felix,', 'Victor,']
>>> pattern = re.compile(r'\w+(?=,)')
>>> pattern.findall(text)
['Felix', 'Victor']
```

匹配那些后面不是跟着空格或.的单词

```
>>> re.findall(r'\w+\b(?![.\s])', text)
['three', 'Felix', 'Victor']
```

分组(子模式):肯定/否定后顾

- 在匹配时要检查**当前位置之前是否满足或者不满足某个模式**。在进行后顾检查时并不会消耗任何字符,即**零宽度匹配**。python实现限制要求**模式的长度为固定长度**,意味着那些重复次数不固定的重复限定符无法使用,选择符中各个子模式长度也要求一样。**不引入捕获组**

(?<=pattern)	零宽度肯定后顾断言(zero-width positive lookbehind assertion), 检查当前位置之前的内容, 要求其满足模式pattern, 在检查时采用零宽度匹配
(?<!pattern)	零宽度否定前瞻断言(zero-width negative lookbehind assertion), 检查当前位置之前的内容, 要求其不与模式pattern匹配, 在检查时采用零宽度匹配

```
>>> text = 'I would rather go out with John Lee than with John Smith or John Baker'
# 匹配前一个单词为John的Smith
>>> re.findall(r'(?<=John\s)Smith', text)
['Smith']
# lookbehind要求之前的模式为固定长度
>>> re.findall(r'(?<=(John|Jonathan)\s)Baker', text)
...
re.error: look-behind requires fixed-width pattern
# 匹配前一个单词不是with的John
>>> re.findall(r'(?<!with\s)John', text)
['John']
```

分组(子模式): 匹配选项

- 通过参数传递匹配选项, `re.compile(pattern, flags)` `re.findall(pattern, text, flags)`
- 在正则表达式中通过`(?flags)`等设置匹配选项, **不引入捕获组**
flags为ilmsux中的一个或者多个字符, 分别对应`re.IGNORECASE` `re.LOCALE` `re.MULTILINE` `re.DOTALL` `re.UNICODE` `re.VERBOSE`

<code>(?flags)</code>	必须出现在整个正则表达式的最开始位置, 给出了匹配时采用的全局选项。
<code>(?flags:pattern)</code>	flags描述的匹配选项仅仅用于该分组的pattern
<code>(?-flags:pattern)</code>	该分组的pattern匹配时去掉flags描述的匹配选项
<code>(?flags1-flags2:pattern)</code>	该分组的pattern匹配时包含flags1但是不包含flags2描述的匹配选项

```
>>> text = 'Cat SCatTeR CATER cAts CatCh'
>>> re.findall(r'\bCat[a-z]*\b', text) # 以Cat开始, 后面为小写字母的单词
['Cat']
>>> re.findall(r'\bCat[a-z]*\b', text, re.I) # 全局大小写无关
['Cat', 'CATER', 'cAts', 'CatCh']
>>> re.findall(r'(?i)\bCat[a-z]*\b', text) # 全局大小写无关
['Cat', 'CATER', 'cAts', 'CatCh']
>>> re.findall(r'\b(?i:Cat)[a-z]*\b', text) # Cat大小写无关
['Cat', 'cAts']
>>> re.findall(r'\b(?-i:Cat)[a-z]*\b', text, re.I) # 全局大小写无关, Cat大小写相关
['Cat', 'CatCh']
```

语法	说明
(?:...)	匹配但不捕获（即对该组不感兴趣）
(?P<groupname>)	为子模式命名，方便对于该组的引用，该分组仍然有编号(捕获组)
(?P=groupname)	在此之前的命名为groupname的子模式匹配的内容
(?=...)	零宽度肯定前瞻断言，检查当前位置之后的内容，要求其满足模式
(?!...)	零宽度否定前瞻断言，检查当前位置之后的内容，要求其不满足模式
(?<=...)	零宽度肯定后顾断言，检查当前位置之前的内容，要求其满足模式
(?<!=...)	零宽度否定后顾断言，检查当前位置之前的内容，要求其不满足模式
(?(id/name)yes no)	如果分组id或name匹配不为空，则要求匹配yes模式，否则要求匹配no模式。no模式是可选的，即(?(id/name)yes)。比如r'(<)?\w+\b(?(1)>)'
(?#...)	表示后面为注释
(?flags)	必须出现在最开始的位置，指定匹配选项，flag为iLmsux中的一个或者多个字符
(?flags1-flags2:pattern)	给出匹配pattern时采用的选项，即设置flags1的选项，去掉flags2中的选项